

Objetivo

Revisar e melhorar completamente a implementação de descoberta e coleta SNMP dos dispositivos Control iD iDFlex.

O problema atual é que o sistema está encontrando poucas métricas via SNMP, enquanto foi comprovado manualmente que o mesmo equipamento responde corretamente a um `snmpwalk` completo.

Equipamento de teste

- Fabricante: Control iD
- Modelo: iDFlex V2
- IP: `10.201.10.120`
- Firmware atualmente reportado pelo SNMP: `5.13.9`
- SNMP: v2c
- Community de teste: `public`

Evidência de que o SNMP funciona

O seguinte comando foi executado no WSL:

```
snmpwalk -v2c -c public 10.201.10.120 1.3.6.1.4.1.49617.1
```

E retornou diversos objetos.

Portanto, o problema não é o SNMP do equipamento. O problema está provavelmente na implementação de descoberta, OID mapping, tratamento de tabelas/índices ou na forma como os resultados são filtrados.

Resultado real obtido no WSL

```
iso.3.6.1.4.1.49617.1.1.1.0 = STRING: "5.13.9"
iso.3.6.1.4.1.49617.1.1.2.0 = STRING: "0G0300/00062F"
iso.3.6.1.4.1.49617.1.1.3.0 = STRING: "1.420000 1.170000 0.730000"
iso.3.6.1.4.1.49617.1.1.4.0 = STRING: "23.436"
iso.3.6.1.4.1.49617.1.1.5.0 = Gauge32: 91991
iso.3.6.1.4.1.49617.1.1.6.0 = STRING: "17-08-2026 15-40-28"
iso.3.6.1.4.1.49617.1.1.7.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.1.8.0 = STRING: "a.st1.ntp.br b.st1.ntp.br"

iso.3.6.1.4.1.49617.1.2.1.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.2.2.0 = Gauge32: 80

iso.3.6.1.4.1.49617.1.3.1.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.3.2.0 = Gauge32: 43200
```

```

iso.3.6.1.4.1.49617.1.3.3.0 = STRING: "timed"

iso.3.6.1.4.1.49617.1.4.1.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.4.2.1 = ""
iso.3.6.1.4.1.49617.1.4.2.2 = STRING: "Full"

iso.3.6.1.4.1.49617.1.5.1.0 = Gauge32: 0
iso.3.6.1.4.1.49617.1.5.2.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.5.3.0 = INTEGER: 1
iso.3.6.1.4.1.49617.1.5.4.0 = INTEGER: 1
iso.3.6.1.4.1.49617.1.5.5.0 = INTEGER: 2

iso.3.6.1.4.1.49617.1.6.1.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.6.2.0 = Gauge32: 5
iso.3.6.1.4.1.49617.1.6.3.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.6.4.0 = INTEGER: 1
iso.3.6.1.4.1.49617.1.6.5.0 = Gauge32: 120
iso.3.6.1.4.1.49617.1.6.6.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.6.7.0 = INTEGER: 2
iso.3.6.1.4.1.49617.1.6.8.0 = INTEGER: 1
iso.3.6.1.4.1.49617.1.6.9.0 = Gauge32: 0

iso.3.6.1.4.1.49617.1.7.1.0 = Gauge32: 2
iso.3.6.1.4.1.49617.1.7.2.0 = ""
iso.3.6.1.4.1.49617.1.7.3.1 = Gauge32: 65793
iso.3.6.1.4.1.49617.1.7.3.2 = Gauge32: 84796
iso.3.6.1.4.1.49617.1.7.4.1 = INTEGER: 1
iso.3.6.1.4.1.49617.1.7.4.2 = INTEGER: 1
iso.3.6.1.4.1.49617.1.7.5.1 = STRING: "2.1.2"
iso.3.6.1.4.1.49617.1.7.5.2 = STRING: "2.1.2"
iso.3.6.1.4.1.49617.1.7.6.1 = Gauge32: 3000
iso.3.6.1.4.1.49617.1.7.6.2 = Gauge32: 3000
iso.3.6.1.4.1.49617.1.7.7.1 = INTEGER: 2
iso.3.6.1.4.1.49617.1.7.7.2 = INTEGER: 2
iso.3.6.1.4.1.49617.1.7.8.1 = INTEGER: 2
iso.3.6.1.4.1.49617.1.7.8.2 = INTEGER: 2
iso.3.6.1.4.1.49617.1.7.9.1 = INTEGER: 2
iso.3.6.1.4.1.49617.1.7.9.2 = INTEGER: 2

```

Requisito 1 — Implementar descoberta recursiva

Não utilizar apenas uma lista fixa de OIDs.

A descoberta deve executar um SNMP WALK/GETBULK recursivo a partir de:

```
1.3.6.1.4.1.49617.1
```

O resultado deve conter TODOS os OIDs retornados pelo equipamento.

O sistema deve ser capaz de descobrir objetos que:

- terminam em `.0`;
- possuem índices;
- fazem parte de tabelas;
- possuem múltiplas instâncias;
- possuem valores `STRING`;
- possuem `INTEGER`;
- possuem `Gauge32`;
- eventualmente possuam outros tipos SNMP.

Não assumir que todo OID termina em `.0`.

Por exemplo, estes objetos são válidos e precisam ser coletados:

```
1.3.6.1.4.1.49617.1.4.2.1
1.3.6.1.4.1.49617.1.4.2.2
```

Também existem tabelas com índices como:

```
1.3.6.1.4.1.49617.1.7.3.1
1.3.6.1.4.1.49617.1.7.3.2
```

Portanto, não filtrar esses objetos por não terminarem em `.0`.

Requisito 2 — Não descartar valores por tipo

O parser SNMP deve preservar:

- OID;
- tipo ASN.1/SNMP;
- valor bruto;
- valor normalizado;
- unidade, quando conhecida;
- índice da tabela, quando existir.

Exemplo:

```
{
  "oid": "1.3.6.1.4.1.49617.1.1.4.0",
  "type": "Gauge32",
  "rawValue": 91991,
  "normalizedValue": 91991
}
```

Não descartar automaticamente `Gauge32`, `INTEGER` ou `STRING`.

Requisito 3 — Separar descoberta de interpretação

Primeiro descobrir:

OID → tipo → valor

Depois interpretar:

OID → nome da métrica → unidade → categoria

Não fazer o contrário.

Se o sistema não conhece o nome de um OID, ele ainda deve armazená-lo como:

Unknown OID

em vez de ignorá-lo.

Requisito 4 — Implementar fallback

Se o sistema possuir uma MIB Control iD instalada, utilizá-la para resolver:

OID → nome
OID → descrição
OID → tipo
OID → unidade

Caso a MIB não esteja disponível, o sistema ainda deve funcionar utilizando o OID numérico.

Requisito 5 — Health Monitoring

Depois da descoberta, identificar e classificar as métricas relevantes para saúde do equipamento.

Prioridade:

Identificação

Firmware
Serial

Performance

Load Average
CPU Usage

Hardware

CPU Temperature

Sistema

Date/Time
NTP Enabled
NTP Server

Rede

Network interface status
Duplex

Não limitar o sistema somente a essas métricas. Se existirem outras métricas relevantes retornadas pelo equipamento, armazená-las para futura classificação.

Requisito 6 — Não assumir que os nomes da documentação estão alinhados ao firmware

O equipamento de teste está em firmware:

5.13.9

A implementação deve validar os OIDs efetivamente retornados pelo dispositivo.

Não assumir que um OID possui determinado significado somente porque uma documentação de outra versão o descreve dessa forma.

Por exemplo, antes de interpretar:

1.3.6.1.4.1.49617.1.1.5.0

como temperatura ou CPU, validar a definição na MIB correspondente.

Requisito 7 — Criar diagnóstico de SNMP

Adicionar uma funcionalidade/log de diagnóstico que mostre:

```
SNMP target
SNMP version
Community
Root OID
Total OIDs encontrados
OIDs descartados
Motivo do descarte
Tempo do WALK
Tempo de resposta
Erros SNMP
```

Exemplo:

```
SNMP Discovery
Target: 10.201.10.120
Version: SNMPv2c
Root: 1.3.6.1.4.1.49617.1

Objects discovered: 47
Objects discarded: 0
Errors: 0
Duration: 183ms
```

Requisito 8 — Comparação com snmpwalk

Criar um teste automatizado que compare a descoberta da aplicação com o resultado equivalente a:

```
snmpwalk -v2c -c public 10.201.10.120 1.3.6.1.4.1.49617.1
```

A aplicação deve conseguir encontrar os mesmos OIDs retornados pelo `snmpwalk`.

O teste não precisa comparar a ordem dos objetos, mas deve comparar o conjunto de OIDs encontrados.

Requisito 9 — Não hardcodar somente o equipamento de teste

A implementação deve continuar genérica.

Não criar código específico apenas para:

```
10.201.10.120
```

ou apenas para:

```
iDFlex V2
```

A árvore:

```
1.3.6.1.4.1.49617.1
```

deve ser configurável por fabricante/modelo/MIB.

Resultado esperado

Depois da implementação, ao cadastrar um iDFlex com:

```
IP: 10.201.10.120  
SNMP: v2c  
Community: public
```

o sistema deve conseguir descobrir aproximadamente o mesmo conjunto de objetos obtido manualmente pelo WSL.

O objetivo NÃO é simplesmente adicionar mais OIDs fixos.

O objetivo é corrigir a implementação de descoberta SNMP para que:

1. faça WALK recursivo;
2. encontre todos os objetos;
3. suporte objetos escalares e tabelas;
4. não descarte tipos SNMP válidos;
5. preserve OIDs desconhecidos;
6. utilize a MIB quando disponível;
7. depois faça o mapeamento semântico das métricas;
8. permita identificar corretamente as métricas de Health do equipamento.

Antes de alterar o código, analise a implementação atual de SNMP e identifique exatamente por que ela está encontrando apenas algumas métricas enquanto o `snmpwalk` encontra dezenas de objetos.

Não faça uma correção superficial adicionando OIDs manualmente. Corrija a estratégia de descoberta.